Secure Multi-Party Aggregation for Flotilla

Complete Tutorial: Procedure, Data Flow, Operations, Extensibility, and Limitations

Flotilla Secure Aggregation Project

2026-08-14

Contents

Introduction

What Flotilla is

Flotilla is a federated-learning (FL) platform built around a single central server (`flo_server.py`, a Flask/waitress REST API driving `FlotillaServerManager`). Clients discover the server and each other over MQTT (heartbeat/advertising only), and the actual FL protocol — sending a client a model to train, collecting its trained weights back, running validation — travels over gRPC, in an *inverted* topology where the server dials out to each client (`EdgeService`: `StartTraining`, `StreamFile`, `InitBench`, `StartValidation`, `Echo`).

Every round, each selected client trains locally and returns its full **plaintext** PyTorch `state_dict` to the server, pickled into a `bytes` field on an insecure gRPC channel. The server's aggregator plugin (by default `aggregator_fedavg.py`, dynamically loaded by `server/load_aggregator.py` from a config string) averages these plaintext updates, weighted by each client's dataset size, and that becomes the next round's global model.

The problem this project solves

That single aggregation server is a **single point of trust**: it is the one place in the entire system where a client's raw, per-round model update is ever materialized in the clear. Anyone who compromises, subpoenas, or simply operates that one server sees every participant's individual contribution — which, depending on the training task, can leak a great deal about a client's private data (this is the well-studied "gradient/update leakage" problem in federated learning).

The solution: MPC-based secure aggregation

Instead of one server, this project introduces a small **cluster of non-colluding party servers** (3, in the concrete deployment built here). Each client **secret-shares** its trained update across all 3 parties instead of sending it in the clear to anyone. The parties then jointly compute — via a Multi-Party Computation (MPC) protocol — the *sum* of all participating clients' (pre-weighted) updates, and reveal **only that final sum**, never an individual client's contribution, back to the (unmodified) training orchestration flow. `flo_server` still drives rounds, sessions, client selection, checkpointing, and validation exactly as before — it just stops being where a client's raw update is ever visible.

The first concrete backend for this is `hpmpc` (a C++20 MPC framework living alongside this repository), using its **Replicated (2,3) secret sharing, 3-party, semi-honest** protocol (`PROTOCOL=2`).

The genericity requirement

A design constraint was set explicitly and is the single most important thing to understand before touching this code: **this must not become "the hpmpc project."** hpmpc's specific integration shape — everything decided at C++ compile time, one static executable per party per protocol per function, a process spawned fresh for every round, communication via flat binary files and raw sockets, no Python bindings, no persistent daemon — must not leak into the core abstraction. A future MPC library might instead offer Python bindings, a long-lived daemon with an RPC interface, Shamir sharing instead of replicated sharing, or a different party count entirely.

Concretely, this is achieved by following **Flotilla's own existing plugin idiom** rather than inventing something new: `server/load_aggregator.py` already dynamically imports `server/aggregation/aggregator_<name>.py` by a config string, and `server_state_manager.py` already does the same for `server/state_manager/<loc>.py`. This project adds two more loaders, `load_sharing_scheme.py` and `load_backend.py`, using the *exact same* importlib-dispatch-by-string pattern. Section 5 goes through the resulting abstract interfaces in detail, and Section 10 walks through, step by step, what adding a genuinely different MPC library behind this interface would involve.

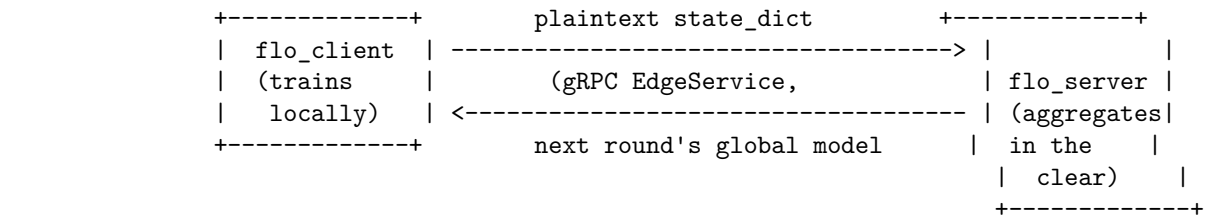
How this document is organized

1. Before/after architecture at a glance
2. The procedure — how this was built, phase by phase
3. Complete catalog of what changed (new files, modified files, why)
4. The core abstractions (the genericity layer)
5. The current data flow, in full detail, for one training round
6. How the system is configured, deployed, and operated day to day
7. The hpmpc backend, in depth (the hardest part of this project)
8. What was actually validated (tests, and a real end-to-end training run)
9. How to add a new MPC backend — a worked, step-by-step guide

10. Current limitations, stated plainly
11. Appendix: command cheat sheet, directory map, glossary

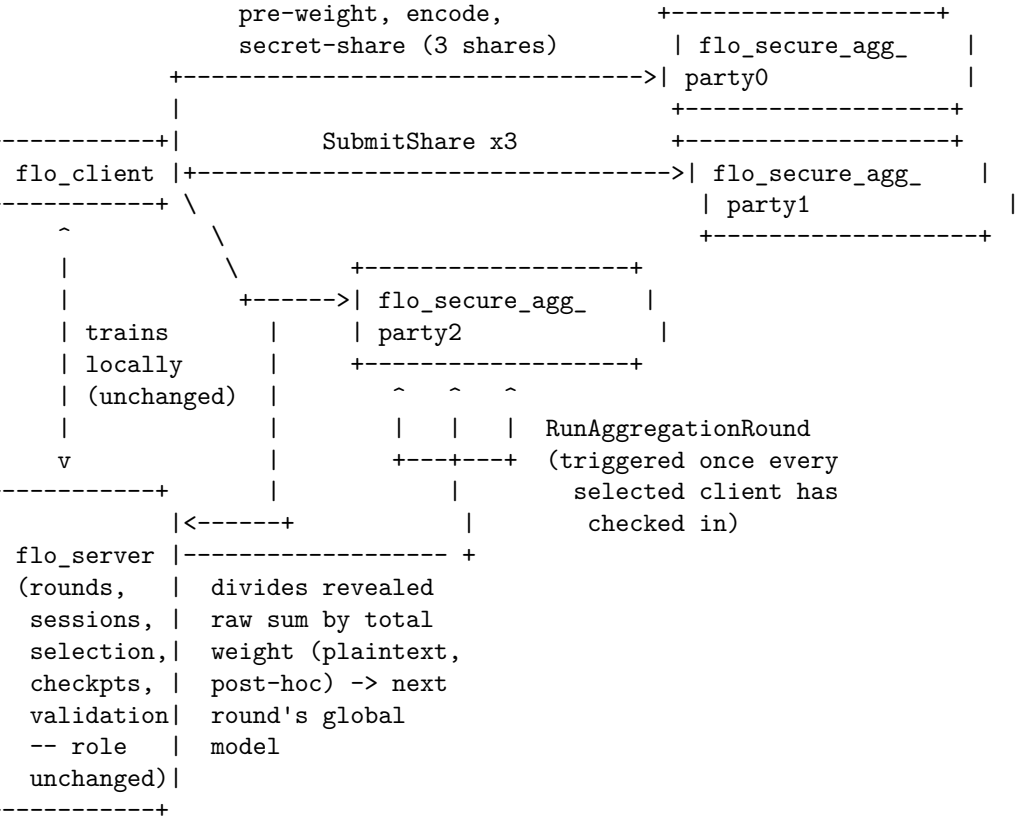
Architecture at a glance

Before



One server. It sees every client's raw update every round. That is the single point of trust being removed.

After



`flo_server` never touches a client's share or its plaintext update. It only ever sees the *final* revealed aggregate — exactly the same shape of value `aggregator_fedavg.py` already hands it today. This is why the change is additive and toggleable rather than a rewrite: everything **above** the aggregator plugin boundary (round sequencing, client selection, checkpointing, validation, the REST API, the client's local training loop) is completely unchanged.

What changed, in one paragraph

A new standalone process type, `flo_secure_agg_party.py`, runs 3 times (one per MPC party) as its own containers/processes — never as a mode flag on `flo_server.py` (see ADR-0001 in Section 2 for why). Clients gain a second outbound gRPC relationship, to the 3 party endpoints, used only to submit shares. A new aggregator plugin, `aggregator_secure_mpc.py`, is loaded via the *existing* `load_aggregator.py` mechanism exactly like `aggregator_fedavg.py` is today, and it fans a round out to the 3 parties instead of averaging plaintext weights it already has. Two small, additive, backward-compatible changes land in existing files (`grpc.proto`, `server_session_manager.py`) to let a client's gRPC response legitimately omit its weights when secure aggregation is on. Everything else — training, validation, checkpointing, the REST API, MQTT discovery — is untouched.

The procedure: how this was built

Development proceeded in five phases, each with its own exit criteria, its own automated tests, and its own documentation, gated on the previous phase's tests passing before starting. This section is the "how it was done" story; Section 3 is the "what changed" catalog; Section 5 onward is the reference material for using and extending the result.

Phase 0 — Foundations

Goal: stand up test infrastructure (none existed in Flotilla before this project — no `tests/` directory, no pytest/unittest usage, no CI) and define every abstract interface with zero behavior, so later phases have a fixed contract to implement against rather than discovering it as they go.

Delivered: `tests/{unit,integration,e2e}/` directory structure, `pytest.ini` with markers `unit`/`integration`/`e2e`/`slow_hp` (so the expensive hpmpc-build tier stays opt-in), `tests/conftest.py` fixtures (a toy `state_dict`, a plaintext-FedAvg reference to diff against), a GitHub Actions workflow running the fast tiers on every push, and the three core ABCs — `SecretSharingScheme`, `SecureAggregationBackend`, `FixedPointCodec` — fully typed and importable but raising `NotImplementedError` everywhere. `docs/secure_aggregation/design.md`, `adr-0001-topology.md`, and a draft `threat_model.md` were written at this point, capturing three decisions locked in with the project owner up front: **3 parties, hpmpc Replicated (2,3) secret sharing**; **semi-honest / honest-majority** threat model; and **single-machine, multi-process/docker-compose** as the dev/test target (see ADR-0001, reproduced in Section 2.1 below).

Phase 1 — Pure-Python sharing math, zero network, zero hpmpc

Goal: prove the actual cryptographic math (secret sharing, reconstruction, the additive homomorphism the whole design leans on) correct in complete isolation, and prove the aggregator plugin's control flow correct against an in-process simulator — before any process boundary or MPC library enters the picture at all.

Delivered: `sharing_schemes/replicated3pc.py` (the real (2,3)-replicated scheme), `sharing_schemes/shamir_stub.py` (a deliberately structurally *different* stub — 5 parties, threshold 3, `NotImplementedError` bodies — added specifically to prove the `SecretSharingScheme` ABC wasn't accidentally shaped around replicated sharing's specifics, e.g. it must not assume `num_parties == 3` anywhere), `load_sharing_scheme.py`, `fixed_point_codec.py` (complete), `backends/backend_simulator.py`'s in-process `run_in_process()` helper (plays all 3 parties inside one Python process/one test, purely to validate the math), and the first cut of `aggregator_secure_mpc.py` with its orchestration client mocked out in tests. Property-based tests (via `hypothesis`) checked the share/reconstruct round trip and the additive-homomorphism property across random tensors.

A real, pre-existing bug was found and fixed here as a side effect of writing tests against `StateManager`: `server/state_manager/inmemory.py`'s `deletebykey` iterated `for k in keys[-1]` (the *characters* of the last path segment) instead of `for k in keys[:-1]` (the parent path segments) —

this silently broke under Flotilla's default in-memory state backend for every existing caller too (`aggregator_fedat.py`, `aggregator_fedasync.py`), not just new code. Fixed with a regression test (`tests/unit/ test_inmemory_state_manager.py`).

Phase 2 — Real 3-process topology, simulator backend

Goal: this is the phase that actually proves the genericity claim. Every layer above the eventual backend boundary — the new proto contract, the party server, client-side share submission, `flo_server`'s orchestration side — is built and fully tested over a **real network, with 3 real processes**, using a pure-Python `SimulatorBackend`. Phase 3 then only has to slot `backend_hpmpc.py` in behind an interface that has *already* been proven generic here, rather than shaping the interface around hpmpc from the start.

Delivered: `src/proto/secure_agg.proto` (new gRPC service, detailed in Section 5.4) plus generated stubs; a small additive change to the existing `grpc.proto` (`InitTrainResponse.model_weights` becomes `optional bytes`, plus a new `bool secure_agg_used`); `party_server.py` (`SecureAggPartyServicer`, the gRPC-facing side of one party process); `flo_secure_agg_party.py` (the new entrypoint, run 3 times); `load_backend.py`; a *real* (no longer mocked) `party_orchestrator_client.py`; `client_secure_agg_manager.py` (client-side share-and-submit logic); the two small guards added to `server_session_manager.py`'s `grpc_train_callback` (Section 3.3); and a networked `SimulatorBackend` (talks to its 2 peers over a small internal gRPC service, `SecureAggPeerService`).

One design refinement happened here, important enough to call out on its own: the `SecureAggregationBackend.run_aggregation_round` signature originally sketched in Phase 0 took a `weights` parameter, intending for the backend itself to compute the weighted sum. That was dropped — multiplying a secret share by a fractional public weight requires a correct *truncation* step under fixed-point arithmetic (a genuine MPC primitive, not something a reference backend should improvise). Instead, **each client pre-weights its own update by its own, always-locally-known, raw dataset size before ever sharing it** — so every backend only ever needs to *sum* shares (always free, local, zero-communication under additive/replicated sharing) and reveal once; the final division by the round's total weight happens in plaintext, after reveal, in `aggregator_secure_mpc.py`, exactly mirroring how `aggregator_fedavg.py` already computes its own weights today. This one decision is what keeps every backend implementation simple and is discussed further in Section 5.2.

At the user's request, the Phase 2 docker-compose end-to-end test was run live (not just written): it surfaced a real numpy-version mismatch (the test venv had an unpinned numpy 2.4.6 against the container's pinned numpy==1.24.3, breaking unpickling of shared numpy arrays across the version boundary) — documented as a latent wire-format fragility in `proto_contract.md` (Section 6.4) rather than silently patched over.

Also fixed here: a race condition in `SimulatorBackend` where a fast-finishing party popped its own completed-round entry before a slower peer could fetch it via `GetFinalShare`, leaving that peer polling forever for an entry that no longer existed. Fixed by never popping (documented as an accepted unbounded-growth limitation of this *reference* backend — see Section 10).

Phase 3 — The hpmpc backend adapter

Goal: get the real hpmpc library — with its very different integration shape — working correctly behind the *exact same* `SecureAggregationBackend` interface Phase 2 already validated, with zero changes above that interface.

This was by far the hardest phase, because hpmpc's native model assumes the *sharer* of a value is one of its own 3 compute parties (there's no built-in notion of an external client pushing a share in), and its own `+` operator for combining shares turned out to do something other than what a naive reading suggests. Section 7 covers this phase's substance in full detail — the share-format derivation, the `operator+` pitfall, the compile-time double-instantiation issue, and how it was all verified without a working native build environment.

Delivered: `hpmpc/programs/functions/ fedavg_secure_aggregation.hpp` (a new hpmpc `FUNCTION_IDENTIFIER=90`),

hpmpc/protocol_executer.hpp's dispatch entry for it, hpmpc/scripts/ build_fedavg_secure_aggregation.sh, backend_hpmpc.py (the HpmpcBackend class), a multi-stage docker/ Dockerfile.secure_agg_party.hpmpc (not yet wired into the default docker-compose topology at this point — that was deliberately deferred to Phase 4 to avoid regressing Phase 2's fast simulator-backed default), and docs/secure_aggregation/ hpmpc_backend.md.

The macOS development host's Xcode Command Line Tools turned out to be missing libc++ headers (confirmed with a minimal #include <algorithm> failing identically outside hpmpc entirely) — rather than repair the host toolchain or install a Homebrew compiler, the decision was made to do the entire hpmpc build and verification inside an Ubuntu Docker container, which is also exactly what the production Dockerfile does.

Phase 4 — Full rollout

Goal: make secure_mpc a fully documented, config-toggled, real alternative to fedavg — wired into the default Docker topology, and validated against an actual model and dataset, not just synthetic test tensors.

Delivered: docker-compose.yaml updated so the 3 secure_agg_party* services now build the real hpmpc-backed image by default; docker/prepare_mnist_data.py (downloads and partitions real MNIST data across 3 clients + 1 validation split); config/ training_config.fedavg.example.yaml and config/ training_config.secure_mpc.example.yaml (identical LeNet5/MNIST configs except for the aggregator block, for direct side-by-side comparison); docs/secure_aggregation/runbook.md and rollout_guide.md; threat_model.md finalized (its residual-risk claims backed by an actual fault-injection test, not just asserted); a fault-injection integration test (tests/integration/ test_secure_mpc_fault_injection.py); and an aggregator-level equivalence test comparing fedavg and secure_mpc outputs directly (tests/integration/ test_fedavg_vs_secure_mpc_equivalence.py).

At the user's explicit choice, validation went all the way to a **full real training run** — MNIST + LeNet5, 3 rounds, 3 real Docker containers running the real compiled hpmpc binaries — rather than stopping at the aggregator-level equivalence test. Section 8 covers the results, including three real pre-existing Flotilla bugs this run surfaced and fixed along the way (Section 3.5), and a live fault-injection/recovery exercise (Section 8.3).

Phase 5 — Genericity stress-test (optional, not started)

Documented as a stretch goal, not required to ship Phases 0–4: swap in a *second*, structurally different hpmpc protocol variant via config alone (no code change above the backend boundary), and/or write a deliberately stub backend_example_daemon.py demonstrating the interface flexes for a persistent-connection/single-RPC-shaped library with no subprocess-per-round model. Section 9 of this document effectively serves as that worked example in written form, even though the code for it has not been built.

What changed: complete file catalog

2.1 The locked-in design decisions (ADR-0001)

Reproduced here because every later decision traces back to these four answers, settled before any implementation began:

1. **3 parties, hpmpc Replicated (2,3) secret sharing (PROTOCOL=2).** The textbook honest-majority scheme most secure-aggregation FL literature assumes; the smallest, most-auditable protocol hpmpc offers; needs no offline preprocessing phase. (hpmpc's 4-party malicious-secure Tetrad protocol was considered and rejected for now — documented future upgrade path, not blocked by anything here.)
2. **Semi-honest / honest-majority threat model.** Parties are assumed to follow the protocol correctly but might passively try to learn secrets; no protection against a party that actively deviates.

3. **A new standalone process type** (`flo_secure_agg_party.py`, run 3 times), never a mode flag on `flo_server.py`. `flo_server`'s responsibilities (session intake, round sequencing, client selection, checkpointing, validation) have nothing to do with running an MPC party's share-buffering/protocol loop, and the two need different lifecycles and network exposure.
4. **Single-machine, multi-process/docker-compose** for dev/test — real TCP sockets between party processes, just co-located on one host. Multi-machine deployment is a config change (different `host` values), not an architecture change.

2.2 New files

Path	Responsibility
`src/flo_secure_agg_party.py`	New process entrypoint; one process per party index; env-var overrides for docker-compose
`src/server/secure_agg/sharing_schemes/base.py`	`PartyShare` dataclass, `SecretSharingScheme` ABC
`src/server/secure_agg/sharing_schemes/replicated3pc.py`	Real (2,3)-replicated additive secret sharing
`src/server/secure_agg/sharing_schemes/shamir_stub.py`	Structurally-different stub, proves ABC genericity
`src/server/secure_agg/load_sharing_scheme.py`	importlib-dispatch loader (same idiom as `load_aggregator.py`)
`src/server/secure_agg/ fixed_point_codec.py`	Float <-> fixed-point-integer codec
`src/server/secure_agg/backends/base.py`	`PartyEndpoint`, `TensorSpec`, `SecureAggregationBackend` ABC
`src/server/secure_agg/backends/backend_simulator.py`	Pure-Python reference backend (in-process helper + networked `SimulatorBackend`)
`src/server/secure_agg/backends/backend_hpmpc.py`	Subprocess + file-IO adapter over compiled hpmpc executables
`src/server/secure_agg/load_backend.py`	importlib-dispatch loader for backends
`src/server/secure_agg/party_server.py`	`SecureAggPartyServicer` — gRPC-facing side of one party process
`src/server/secure_agg/party_orchestrator_client.py`	`flo_server`-side helper: fan a round out to all parties, collect the reveal
`src/server/aggregation/aggregator_secure_mpc.py`	New aggregator plugin, loaded via the existing `load_aggregator.py`
`src/client/client_secure_agg_manager.py`	Client-side: encode, share, submit to all party endpoints
`src/proto/secure_agg.proto` (+ generated _pb2.py/_pb2_grpc.py)	New gRPC service contract
`src/proto/run_secure_agg.sh`, `run3_secure_agg.sh`	Proto codegen helper scripts
`config/secure_agg_party_config.yaml`, `src/config/secure_agg_party_config.yaml`	Per-party config template
`config/ training_config.fedavg.example.yaml`, `training_config.secure_mpc.example.yaml`	Matched example session configs for side-by-side comparison
`docker/Dockerfile.secure_agg_party`	Simulator-backed party image (fast dev loop, no C++ build)
`docker/ Dockerfile.secure_agg_party.hpmpc`	Multi-stage image: compiles real hpmpc binaries, then a slim runtime
`docker/secure_agg_party_entrypoint.sh`	Container entrypoint for party processes
`docker/ sample_docker_secure_agg_party_run.sh`	Standalone (non-compose) run helper

Path	Responsibility
`docker/prepare_mnist_data.py`	Downloads/partitions real MNIST data for the e2e validation run
`hpmpc/programs/functions/fedavg_secure_aggregation.hpp`	New hpmpc `FUNCTION_IDENTIFIER=90`: sum-then-reveal
`hpmpc/scripts/build_fedavg_secure_aggregation.sh`	Builds all 3 party executables for the new function
`docs/secure_aggregation/*.md`	Design, ADR, threat model, topology, proto contract, sharing-scheme math, hpmpc backend, runbook, rollout guide (this tutorial included)
`tests/{unit,integration,e2e}/...`	All automated test infrastructure (none existed before this project)
`pytest.ini`, `requirements-dev.txt`, `.github/workflows/tests.yml`	Test tooling and CI

2.3 Modified files, and exactly what changed in each

Path	Change
`src/proto/grpc.proto`	`InitTrainResponse.model_weights` -> `optional bytes` (was a plain required field); new `bool secure_agg_used = 6`. Purely additive — when `secure_agg_used` is unset/false, behavior is byte-for-byte identical to before.
`src/server/server_session_manager.py`	Two guards in `grpc_train_callback` (around line 672 and 702): `local_model_wts = pickle.loads(response.model_weights) if response.HasField("model_weights") else None`, and the subsequent `training_state.put(...weights...)` is skipped when `local_model_wts is None`.
`src/client/client_grpc_manager.py`	`StartTraining` branches on `secure_aggregation_config["enabled"]`: when on, it calls `client_secure_agg_manager.share_and_submit(...)` instead of pickling `model_weights` into the response, and sets `secure_agg_used=True`.
`src/client/client_manager.py`	Reads a new `secure_aggregation` block from `client_config.yaml` (default `{"enabled": False}` if absent) and passes it through to `ClientGRPCManager`.
`docker/docker-compose.yaml`	Adds `secure_agg_party0/1/2` services (Phase 2: simulator-backed; Phase 4: switched to build `Dockerfile.secure_agg_party.hpmpc` by default) plus an internal `secure-agg-network`; Phase 4 additionally adds `flo_server`/`flo_client0/1/2` services (previously not part of compose) specifically to make the real end-to-end validation run reproducible.

Path	Change
src/config/client_config.yaml	New secure_aggregation: block (enabled, sharing_scheme, fixed_point, party_endpoints, submission_timeout_s); also, along the way, a **stale personal dev path** in datasets_dir_path (a leftover /home/.../fedml-ng/src/data) was fixed to /src/data, unrelated to secure aggregation but discovered while wiring up the real Docker run.
config/training_config.yaml	New aggregator_args shape documented inline for the secure_mpc aggregator (sharing_scheme, num_parties, party_endpoints, fixed_point, round_timeout_s, verify_party_agreement).

2.4 Real, pre-existing bugs found and fixed along the way

None of these are secure-aggregation logic — they are latent bugs in Flotilla's existing plaintext code path, surfaced either by writing the first-ever test suite against it, or by actually running a full real training session end to end for the first time. Each is called out explicitly (rather than silently folded in) because they affect the **plaintext fedavg path too**, not just secure aggregation:

1. **server/state_manager/inmemory.py, deletebykey** — iterated for k in keys[-1] (characters of the last path segment) instead of for k in keys[:-1] (the parent path segments). Broke under the in-memory state backend for every caller, including the pre-existing aggregator_fedat.py/aggregator_fedasync.py — it only ever "worked" by accident under the Redis backend, whose iteration semantics happened to tolerate it differently. Found in Phase 1 while writing the first regression test for StateManager.
2. **server/aggregation/aggregator_fedavg.py** — finished_clients = aggregator_state.keys() returns a non-subscriptable dict_keys view under the in-memory backend; finished_clients[0] then crashes. Only "worked" under Redis, whose .keys() happens to return a real list. Fixed to list(aggregator_state.keys()). Surfaced by the new tests/integration/test_fedavg_vs_secure_mpc_equivalence.py in Phase 4.
3. **client/client_file_manager.py, get_dataset_details** — summary_path = path.split(".")[1] + "_summary.data" only produces a correct path for "./relative/path.ext"-shaped inputs; for an absolute path like /src/data/MNIST/x.pth it silently grabbed just "pth", so the summary file was never found and the function returned None. Fixed to os.path.splitext(path)[0] + "_summary.data". Surfaced during the live Phase 4 Docker run (clients mount data at absolute paths).
4. **client/client_grpc_manager.py (this project's own Phase 2 code)** — assumed get_dataset_details(...)["metadata"]["num_items"] (the nested shape the *server*-side current_dataset_detail structure uses), but the client-side helper actually returns a flat dict ({"num_items": ..., ...} directly, no "metadata" wrapper — the format utils/get_data_summary.py produces). Fixed to get_dataset_details(...)["num_items"]. Also surfaced during the Phase 4 live run, compounded with bug 3 above.

Each of these has a dedicated regression test now (tests/unit/ test_inmemory_state_manager.py, tests/unit/test_aggregator_fedavg.py, tests/unit/test_client_file_manager.py).

2.5 Post-rollout enhancement: dataset sizes are also secret-shared

After Phase 4 shipped, a gap was closed: originally, flo_server learned every client's dataset size in the clear (via the pre-existing plaintext InitBench/StartTraining RPCs — the same mechanism fedavg still uses today) and sent each client's weight *fraction* to the parties on RunAggregationRoundRequest.client_weights. That field is now gone. In secure_mpc sessions, each client *also* secret-shares its raw dataset size, under a reserved DATASET_SIZE_LAYER_NAME pseudo-layer (server/secure_agg/constants.py) — summed and revealed by the exact same generic backend mechanism already used for model-weight layers, with **zero backend**

code changes required (see Section 5.6 and Section 4.3's addendum on why). `aggregator_secure_mpc.py` now pops the revealed total and uses it as the division denominator; neither it nor any party ever learns an individual client's dataset size in `secure_mpc` mode.

New/changed for this enhancement: `src/server/secure_agg/constants.py` (new); `client_secure_agg_manager.py` (also shares the pseudo-layer); `secure_agg.proto` (`client_weights` field removed, `reserved 4`, stubs regenerated); `party_orchestrator_client.py` (`run_round` now takes `client_ids`, not `client_weights`); `aggregator_secure_mpc.py` (pops and validates the revealed total instead of computing it from `training_state`); `backends/base.py`'s docstring (addendum); plus `tests/unit/test_client_secure_agg_manager.py` (new) and updates across the existing aggregator/integration/e2e tests. See `threat_model.md` for the updated protected/not-protected breakdown.

The core abstractions (the genericity layer)

This is the part of the system a future contributor needs to understand before touching anything — everything else in this document is either "how the current concrete pieces implement these interfaces" or "how to add a new concrete piece behind them."

4.1 `SecretSharingScheme` — pure math, zero I/O

`src/server/secure_agg/ sharing_schemes/base.py`:

```python
@dataclass(frozen=True)
class PartyShare:
    party_index: int
    payload: Any            # scheme-specific, must be picklable

class SecretSharingScheme(ABC):
    scheme_id: str
    num_parties: int
    reconstruction_threshold: int

    def share(self, plaintext_fixedpoint: np.ndarray, rng) -> list[PartyShare]:
        """Split a fixed-point tensor into num_parties PartyShares."""

    def reconstruct(self, shares: dict[int, PartyShare]) -> np.ndarray:
        """Combine enough shares back into plaintext. Tests/debug only --
        real MPC parties never call this in production."""

    def add(self, a: PartyShare, b: PartyShare) -> PartyShare:
        """Locally (no communication) add two same-party shares."""
```

No network, no dependency on any MPC library — implementations are testable without any backend installed at all. The concrete `Replicated3PCScheme` (Section 4.5) and the deliberately-different `shamir_stub.py` (5 parties, threshold 3, `NotImplementedError` bodies) both satisfy this ABC, which is exactly the point: the stub exists purely to catch an ABC design that accidentally assumes 3 parties or replicated-sharing specifics anywhere.

4.2 `FixedPointCodec` — float <-> fixed-point integer

`src/server/secure_agg/ fixed_point_codec.py`. Deliberately independent of any sharing scheme (mirrors hpmpc's own separation of `Additive_Share` from `FloatFixedConverter`):

```python
class FixedPointCodec:
    def __init__(self, bitlength: int, frac_bits: int): ...
    def encode(self, plaintext: np.ndarray) -> np.ndarray:
        """round(x * 2**frac_bits); raises OverflowError rather than
        silently clipping/wrapping a value that doesn't fit."""
    def decode(self, fixedpoint: np.ndarray) -> np.ndarray:
        """fixedpoint / 2**frac_bits, back to float64."""
```

`bitlength`/`frac_bits` must match whatever a given backend's numeric assumptions are — for the hpmpc backend, they must equal the compile-time `BITLENGTH`/`FRACTIONAL` macros baked into the party executables. This is a cross-language config-consistency hazard (there is no automatic way to keep a Python config value and a C++ compile-time constant in sync) — see Section 7.6 for the explicit startup check that catches drift.

Raising `OverflowError` rather than silently corrupting a value is a deliberate choice: a loud crash during development is a far better failure mode than a silently wrong model weight propagating through training.

4.3 `SecureAggregationBackend` — the heart of the genericity layer

`src/server/secure_agg/backends/base.py`:

```python
@dataclass(frozen=True)
class PartyEndpoint:
    party_index: int
    host: str
    port: int

@dataclass(frozen=True)
class TensorSpec:
    layer_name: str
    shape: tuple
    dtype: str

class SecureAggregationBackend(ABC):
    backend_id: str
    party_index: int
    num_parties: int

    async def start(self, peer_endpoints: list[PartyEndpoint]) -> None:
        """One-time setup after construction, before any round runs."""

    async def run_aggregation_round(
        self,
        round_id: str,
        shares: dict[str, dict[str, PartyShare]],  # client_id -> layer -> this party's share
        tensor_specs: dict[str, TensorSpec],
        timeout_s: float,
    ) -> "OrderedDict[str, torch.Tensor]":
        """Run one MPC round against the other num_parties-1 peers; return
        this party's PLAINTEXT reveal of the SUM of the given shares (not a
        weighted average). Every honest party must return an identical
        result."""

    async def stop(self) -> None:
        """Release any resources acquired in start()."""
```

The interface says **nothing** about *how* peer communication happens during a round, and that silence is

deliberate and load-bearing: hpmpc opens its own raw TCP sockets between spawned binaries; the reference `SimulatorBackend` talks to peers over a small internal gRPC call; a hypothetical daemon-backed library might issue one RPC to its own persistent process that's already running. None of that shows up in the method signatures above, which is exactly what lets structurally different libraries implement the same interface without `party_server.py`, `aggregator_secure_mpc.py`, or the client ever needing to know or care which shape a given backend uses.

Notice there is deliberately **no `weights` parameter**. See Section 1.3 — Phase 2 — for why: weighting happens client-side, before sharing, so every backend only ever does free local addition, never a fractional scalar-multiply-under-secret-sharing (which would need a real truncation protocol to do correctly).

4.4 The two loaders

`load_sharing_scheme.py` and `load_backend.py` both follow the *exact* importlib-dispatch-by-config-string idiom Flotilla already uses for `load_aggregator.py` and `server_state_manager.py`'s backend loader:

```
def load_backend(id, backend):
    module_name = f"server.secure_agg.backends.backend_{backend}"
    module = importlib.import_module(module_name)
    return module    # caller does module.BACKEND_CLASS(...)
```

Every backend module exposes its concrete class as a module-level `BACKEND_CLASS` attribute (mirrored by `SCHEME_CLASS` for sharing schemes). This is the *entire* extension mechanism — adding a new backend or sharing scheme means adding one new file with the right module-level name, nothing else. Section 9 walks through this concretely.

4.5 The concrete Phase-1 sharing scheme: `Replicated3PCScheme`

A secret `x` (a fixed-point integer) is split into three random components `c0, c1, c2` with `c0 + c1 + c2 === x (mod 2**bitlength)`. Party `i` is handed the pair `(c_i, c_{i+1 mod 3})`:

Party	Holds
0	`(c0, c1)`
1	`(c1, c2)`
2	`(c2, c0)`

No single party's pair reveals anything about `x`; any 2 of the 3 parties' pairs together cover all three components and can reconstruct `x`. This is the standard textbook (2,3)-replicated secret sharing scheme and is exactly the layout hpmpc's `PROTOCOL=2` expects.

The property the whole design leans on is that this scheme is **additively homomorphic under local, zero-communication addition**: if party `i` holds `share(a)_i` and `share(b)_i`, adding the two pairs component-wise gives a valid share of `a + b`, with no network round needed. This is why summing N clients' secret-shared updates costs **zero** network rounds — each party locally sums its own share of every client's update, and only the *final* sum needs a reveal round, regardless of how many clients participated.

Fixed-point rounding error: `encode()` rounds to the nearest `2**-frac_bits` tick, so each value carries at most half a tick of error; summing N clients' weighted updates accumulates at most `N * 2**-frac_bits / 2` in the worst case (linear in N, not compounding). With the default `frac_bits: 13`, resolution is `2**-13 approx. 1.2e-4` — comfortably below typical float32 model-weight precision.

The current data flow, step by step

This section walks one training round through the entire secure-aggregation path, end to end, referencing exactly which file and function does each step. It assumes `secure_aggregation.enabled: True` on the client and `session_config.aggregator: secure_mpc` on the server (Section 6.2 covers the config surfaces that set this).

5.1 Client trains, exactly as before

`flo_server` dials the client's `EdgeService.StartTraining` RPC (unchanged — `client_grpc_manager.py`'s `StartTraining`). The client trains locally via `Client.Train(...)` (unchanged — `client_trainer.py`/`client.py` are not touched by this project at all) and produces a plaintext `state_dict` exactly as it always has.

5.2 Client pre-weights, encodes, and secret-shares its update

Instead of pickling `model_weights` into the gRPC response, `client_grpc_manager.py`'s `StartTraining` sees `secure_aggregation_config["enabled"] == True` and calls `client_secure_agg_manager.share_and_submit(...)`:

1. Look up this client's own raw dataset size via `get_dataset_details(dataset_ path)["num_items"]` (a flat dict produced by `utils/get_data_summary.py` — see the callout in Section 2.4, bug 4, for a shape mismatch that bit this exact line during development).
2. For every layer in the `state_dict`: multiply the tensor by that dataset size (`weighted = tensor * dataset_size`), then `FixedPointCodec.encode()` it into fixed-point integers.
3. `Replicated3PCScheme.share()` splits each encoded tensor into 3 `PartyShare` objects (one per party index).
4. Also secret-share the client's **raw** (not pre-weighted) dataset size itself, under the reserved `DATASET_SIZE_LAYER_NAME` pseudo-layer (`server/secure_agg/constants.py`) — same codec, same sharing scheme, treated as just another layer. This is what lets the party cluster sum every checked-in client's dataset size and reveal only the round's *total*, later, without flo_server or any party ever learning an individual client's dataset size (see Section 10's "protected" list).
5. Build one `SubmitShareRequest` per party endpoint, each carrying only *that* party's share of every real layer plus the dataset-size pseudo-layer (`TensorShare.share_payload = pickle.dumps(party_share.payload)`), and send it via `SecureAggPartyServiceStub.SubmitShare`.

The response's `InitTrainResponse` sets `secure_agg_used=True` and **omits** `model_weights` entirely (it's now `optional bytes` in `grpc.proto` specifically to allow this).

5.3 Each party buffers the incoming share

`party_server.py`'s `SecureAggPartyServicer.SubmitShare` (running inside one of the 3 `flo_secure_agg_party` processes) validates the client's declared `sharing_scheme` matches this party's own configured scheme (rejecting the submission with a clear error otherwise — a config-mismatch guard, not a security boundary), then writes the shares into this party's own `StateManager` instance, keyed `f"{round_id}.shares.{client_id}"`. This reuses the *exact same* pluggable `StateManager` abstraction the rest of Flotilla already uses for session state (in-memory or Redis) — a party's share buffer gets Redis-backed persistence for free, with zero new backend code.

5.4 `flo_server` tracks check-ins, never touching a share

`server_session_manager.py`'s `grpc_train_callback` receives the client's response as usual. Because `response.HasField("model_weights")` is now `False` for a secure-agg client, `local_model_wts` becomes `None` and the subsequent `training_state.put(...weights...)` is skipped — this is the *entire* extent of what changed in the pre-existing plaintext-path code. `server/load_aggregator.py` has already loaded `aggregator_secure_mpc.py` for this session (because `session_config.aggregator: secure_mpc`), and its `aggregate(...)` function is called with the *same* signature every aggregator plugin gets. It:

1. Marks this client as `checked_in` in its own `aggregator_state` (unrelated to the party servers' separate share buffers).
2. Once every currently-selected, currently-active client has checked in, proceeds to trigger the round; otherwise returns `None` (exactly the "not ready yet" contract every aggregator plugin uses).

Unlike `aggregator_fedavg.py`, this function never looks up any client's dataset size from `training_state` — it doesn't know it, and doesn't need to (see Section 5.6). All it needs at this point is the list of checked-in `client_id`s to tell the parties which buffered shares to include.

5.5 `flo_server` triggers the round; every party runs its backend

`aggregator_secure_mpc.py` calls `party_orchestrator_client. run_round(...)`, which fans a `RunAggregationRound` RPC out to all 3 party endpoints concurrently (a small thread pool — deliberately synchronous, since `aggregate()` itself is called synchronously from a non-async context; see `party_orchestrator_client.py`'s module docstring). Each party's `SecureAggPartyServicer. RunAggregationRound`:

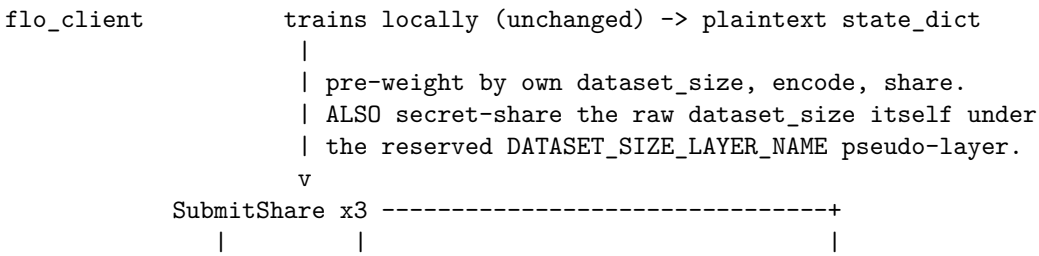
1. Checks every expected `client_id` actually has a buffered share for this `round_id` — if any is missing, it fails immediately with a clear error rather than hanging (a live MPC round genuinely cannot proceed with a client's share missing).
2. Calls its configured `SecureAggregationBackend. run_aggregation_round(...)` — this is where the *actual* MPC protocol execution happens, and where the simulator and hpmpc backends diverge completely in implementation while presenting an identical return contract.
3. Clears its share buffer for this round (success or failure) and returns the revealed plaintext `OrderedDict[str, torch.Tensor]` — the **raw weighted sum** for every real model layer, plus the revealed `DATASET_SIZE_LAYER_NAME` entry (the round's total dataset size), none of it yet divided by that total.

`run_round` requires **all** parties to respond successfully (unlike offline reconstruction, which only needs 2 of 3 shares — the *live* protocol genuinely needs every party online and participating). If `verify_party_agreement` is enabled (default), it also cross-checks every party revealed the identical plaintext before returning — a correctness/ liveness sanity check for catching bugs, not a security guarantee.

5.6 `flo_server` pops the revealed total and divides, exactly as `fedavg` would

Back in `aggregator_secure_mpc.py`, `raw_sum.pop(DATASET_SIZE_LAYER_NAME)` pulls out the revealed total dataset size (decoded, rounded to the nearest integer, and checked `> 0` — a round that revealed a non-positive total fails cleanly rather than dividing by it). Every remaining real layer is then divided, in plaintext, by that total — the *only* "weighting" arithmetic secure aggregation still needs to do outside the MPC protocol, trivial because it happens after reveal. Unlike Phases 0–4's original design, **flo_server never independently computes this total from plaintext dataset sizes — it only ever sees the value the MPC round itself revealed.** The result (with the pseudo-layer stripped out) is handed back to `server_session_manager.py` exactly like `aggregator_fedavg.aggregate()`'s return value always has been, and training proceeds — checkpointing, validation, and the next round's `StartTraining` dial-out are completely unaware secure aggregation was ever involved.

5.7 The whole thing as one diagram



```
          v         v                                   v
flo_secure_agg_party0   flo_secure_agg_party1   flo_secure_agg_party2
          ^         ^                                   ^
          |         |                                   |
          +--- RunAggregationRound (triggered by flo_server, --------+
               carries only client_ids -- no per-client weight)
                              |
                              v
                        flo_server (aggregator_secure_mpc.py)
                  pops the revealed DATASET_SIZE_LAYER_NAME total, divides every
                  other revealed layer by it (plaintext, post-hoc)
                         -> global_model (plaintext, same shape as today)
```

`flo_server` sits only at the bottom: it triggers rounds and receives the final plaintext aggregate, plus the round's total dataset size (used only as a division denominator, then discarded). It is never in the path a client's share — or dataset size — travels, and it never sends any per-client weight to the parties.

How the system is configured, deployed, and operated

6.1 The two independent toggles

Secure aggregation needs **both** of the following set consistently — they are independent config surfaces because they're read by different processes at different times:

1. **Server side**: `session_config.aggregator: secure_mpc` in the training config submitted via `flo_session.py` (see `config/ training_config.secure_mpc.example.yaml`). This selects `aggregator_secure_mpc.py` for that specific session.
2. **Client side**: `secure_aggregation.enabled: True` in each client's `client_config.yaml`, plus a matching `party_endpoints` list. This is read once at client process startup, not per session.

These must agree. A client with `enabled: True` always secret-shares its update and omits `model_weights`, regardless of which aggregator the session picked — pairing it with `aggregator: fedavg` breaks that session (`aggregator_fedavg.py` expects real weights, not `None`). A `secure_mpc` session with `enabled: False` clients will simply never receive any shares and hang waiting for check-ins that never come. Toggling either requires restarting that process — the server picks up a new `aggregator` per session submission automatically, but a client's `enabled` flag needs a client-container restart.

6.2 Config file reference

`client_config.yaml` (`src/config/client_config.yaml`), new block:

```yaml
secure_aggregation:
  enabled: False              # default -- byte-for-byte unchanged plaintext path
  sharing_scheme: replicated3pc
  fixed_point:
    bitlength: 64
    frac_bits: 13
  party_endpoints:
    - {party_index: 0, host: secure_agg_party0, port: 50100}
    - {party_index: 1, host: secure_agg_party1, port: 50101}
    - {party_index: 2, host: secure_agg_party2, port: 50102}
  submission_timeout_s: 30
```

`training_config.yaml`, `session_config` block:

```yaml
session_config:
  aggregator: secure_mpc
  aggregator_args:
    sharing_scheme: replicated3pc
    num_parties: 3
    party_endpoints: [{host: secure_agg_party0, port: 50100}, ...]
    fixed_point: {bitlength: 64, frac_bits: 13}
    round_timeout_s: 120
    verify_party_agreement: true   # sanity cross-check, not a security guarantee
```

secure_agg_party_config.yaml (config/secure_agg_party_config.yaml), one per party (or one shared file with env-var overrides — see below):

```yaml
party_index: <0, 1, or 2>
num_parties: 3
bind_host: 0.0.0.0
bind_port: <control_plane_port>     # SecureAggPartyService
backend_port: <backend_wire_port>   # this backend's own inter-party protocol port
peers:
  - {party_index: <peer_a>, host: <host_a>, backend_port: <port_a>}
  - {party_index: <peer_b>, host: <host_b>, backend_port: <port_b>}
sharing_scheme: replicated3pc
fixed_point: {bitlength: 64, frac_bits: 13}
backend:
  type: simulator   # simulator | hpmpc
  simulator: {bind_host: 0.0.0.0}
  hpmpc: {executable_dir: <path>, tmp_dir: <scratch_dir>}
state:
  state_location: inmemory   # or redis
  state_hostname: <redis_ip_if_used>
  state_port: <redis_port_if_used>
grpc_workers: 8
```

`fixed_point` must match **exactly** across all three files, and — for the hpmpc backend — the compiled binaries too (Section 7.6 covers the automated check that catches drift here).

The handful of fields that legitimately differ per party (`party_index`, `bind_port`, `backend_port`, `peers`, `backend.type`) can also be set via environment variables (`PARTY_INDEX`, `BIND_PORT`, `BACKEND_PORT`, `PEERS_JSON`, `BACKEND_TYPE`) so the *same* checked-in config file is reused across all 3 containers — see `flo_secure_agg_party.py`'s `_apply_env_overrides()`. `PEERS_JSON` exists specifically because `peers` is a YAML list, which the `sed`-based patching Flotilla's other entrypoint scripts use cannot edit safely.

6.3 Processes and network relationships

Process	Entrypoint	Count	Role
`flo_server`	`src/flo_server.py`	1	Unchanged control-plane orchestrator. Never sees a client's raw or shared update.

Process	Entrypoint	Count	Role
`flo_client`	`src/flo_client.py`	N	Trains locally (unchanged). If enabled, secret-shares its update and submits directly to the 3 party processes instead of sending plaintext weights to `flo_server`.
`flo_secure_agg_party`	`src/flo_secure_agg_party.py`	3	One process per MPC party. Buffers shares, runs the configured backend, reveals the aggregate back to `flo_server`.

```
flo_client ---(gRPC EdgeService, unchanged)---> flo_server
flo_client ---(gRPC SecureAggPartyService.SubmitShare)---> flo_secure_agg_party {0,1,2}
flo_server ---(gRPC SecureAggPartyService.RunAggregationRound)---> flo_secure_agg_party {0,1,2}
flo_secure_agg_party i ---(backend-specific wire protocol)---> flo_secure_agg_party j
```

`flo_server` never talks to `flo_client` about shares, and never talks to any party process except to trigger/collect a round's result — it has no share-transport role at all. Party-to-party traffic (`backend_port`) is entirely backend-private: for the simulator backend it's a small internal gRPC service; for hpmpc it's that library's own raw TCP sockets between freshly-spawned binaries. Neither choice requires any change above the backend boundary.

6.4 Docker Compose topology

`docker/docker-compose.yaml` defines 3 `secure_agg_party0/1/2` services, each on **two** networks: the shared `flotilla-network` (same network `redis`/`mqtt5` use — this is how `flo_client`/`flo_server` reach a party's control-plane port) and an `internal: true` `secure-agg-network` that only the 3 party containers join (their private inter-party wire port has no business being reachable from `flo_client`/`flo_server`, and `internal: true` enforces that at the Docker network level, not just by convention).

By default (Phase 4), the 3 services build `docker/ Dockerfile.secure_agg_party.hpmpc` — a multi-stage build that compiles the real hpmpc executables in one stage (`additional_contexts: {hpmpc_src: ../../hpmpc}`, since hpmpc lives in a sibling directory outside this repo's own build context) and copies just the binaries into a slim runtime stage. For a fast dev loop with no C++ toolchain required at all, switch a service's `dockerfile:` back to `docker/Dockerfile.secure_agg_party` (the single-stage, simulator-backed image from Phase 2) and set `BACKEND_TYPE: simulator`.

`flo_server`/`flo_client0/1/2` are **not normally** part of docker-compose (operators run them via `sample_docker_server_run.sh`/ `sample_docker_client_run.sh` against a real cluster instead) — they're included in `docker-compose.yaml` specifically to make the Phase 4 real end-to-end validation run reproducible on one machine.

6.5 Bringing the cluster up, health-checking, tearing down

```
cd flotilla/docker
docker compose up -d --build secure_agg_party0 secure_agg_party1 secure_agg_party2
```

The first build compiles hpmpc from scratch (a few minutes); subsequent builds are fast via Docker layer caching unless `hpmpc/` source changes.

Health check (expect `True hpmpc` — or `True simulator` — for all 3 ports):

```
python3 -c "
import grpc
import proto.secure_agg_pb2 as pb2
import proto.secure_agg_pb2_grpc as pb2_grpc
for port in (50100, 50101, 50102):
    ch = grpc.insecure_channel(f'localhost:{port}')
    r = pb2_grpc.SecureAggPartyServiceStub(ch).HealthCheck(pb2.HealthCheckRequest(), timeout=5)
    print(port, r.ready, r.backend_id)
"
```

Tear down: `docker compose down -v`.

6.6 Running a full example session

```
cd flotilla/docker
python3 prepare_mnist_data.py          # one-time: download + partition MNIST
docker compose up -d --build            # redis, mqtt5, 3 parties, flo_server, flo_client0/1/2
pip install requests pyyaml             # flo_session.py's only host-side deps
python3 ../flo_session.py ../config/training_config.secure_mpc.example.yaml \
    --federated_server_endpoint localhost:12345
docker compose logs -f flo_server       # watch progress
```

For the plaintext comparison run, flip each client's `secure_aggregation.enabled` to `False` and submit `training_config.fedavg.example.yaml` instead — the two configs are identical in model, dataset, and hyperparameters, so any difference in the resulting accuracy/loss curves beyond the documented fixed-point rounding tolerance would indicate a real bug.

6.7 Debugging a hung or failed round

A round can legitimately *fail* (bounded by `round_timeout_s`), but should never hang forever. If training seems stuck:

1. **Check each party's health** (Section 6.5). A crashed/unreachable party is the most common cause — the live MPC round genuinely needs all 3 parties (see Section 10's "no fault tolerance" limitation), so training cannot progress until it's back.
2. **Check `flo_server`'s logs** for `fedserver.aggregator. secure_mpc.exception` — `aggregator_secure_mpc.py` logs which party failed and why before returning `None` for that round; `flo_server` keeps retrying on subsequent client check-ins, it does not treat one failed round as fatal.
3. **Check a party's own logs** for `fedparty.run_round.missing_shares` — means `flo_server` triggered the round before every expected client's share had arrived at that party; usually a still-training client, or a client whose `SubmitShare` call to one or more parties failed (check the client's logs for `fedclient.secure_agg. submit_share.rejected`).
4. **hpmpc-specific**: `fedparty.run_round.exception` on an hpmpc-backed party most often means either a config mismatch caught at party *startup* (Section 7.6 — check logs right after the container starts, not mid-round) or the spawned executable itself failing/timing out (the exception message includes its stdout/stderr).

If you change `fixed_point.bitlength`/`frac_bits`, the compiled hpmpc executables must be rebuilt to match, or the party will refuse to start:

```
cd hpmpc && scripts/build_fedavg_secure_aggregation.sh <bitlength> <frac_bits>
cd ../flotilla/docker && docker compose up -d --build secure_agg_party0 secure_agg_party1 secure_agg_pa
```

The hpmpc backend, in depth

This section covers the hardest engineering problem in this project: making hpmpc — a library designed around 3 mutually-trusting compute parties secret-sharing values *among themselves* — accept shares an **external** client (a Flotilla `flo_client`, not one of hpmpc's own compute parties) already computed in pure Python.

7.1 Protocol choice and build flags

`PROTOCOL=2` (Replicated 3PC), `FUNCTION_IDENTIFIER=90` (the new function described below), `BITLENGTH=64`, `FRACTIONAL=13`, and three portability choices worth explaining:

- `DATTYPE=64` (with `BITLENGTH=64`) routes to hpmpc's portable `core/arch/STD.h` (plain `uint64_t`, no SIMD intrinsics) instead of `AVX.h/SSE.h`, and gives `vectorization_factor = 1` — no packing to reason about.
- `RANDOM_ALGORITHM=0` selects the portable xorshift PRNG instead of the default AES-based one, which uses x86 AES-NI intrinsics and does not build on ARM (relevant since development happened on Apple Silicon).
- `USE_SSL=0` skips OpenSSL transport encryption between hpmpc's own sockets, avoiding cert provisioning for local testing.

These are dev/test-friendly choices, not production-hardened ones — see Section 10 for what a real deployment should change.

7.2 The share-format problem

hpmpc's native `Replicated_Share(x, a)` layout (`protocols/3-PC/replicated/ replicated_template.hpp`) is a **different** (though algebraically related) representation from the `(c_j, c_{j+1})` pair `replicated3pc.py` uses. Tracing `prepare_reveal_to_all()`/ `complete_Reveal()` gives the reveal invariant every party's `(x, a)` pair must satisfy: `a_{p+1} = x_p - secret` for every ring position `p`. Critically, `x_p` can be **any** value — even independently random per party — as long as `a_p` is computed consistently with it, which is what makes a **per-party-local** conversion possible, needing zero coordination between parties:

```
x_j = c_j
a_j = -(c_j + c_{j+1})   (mod 2**bitlength)
```

This was verified two ways: algebraically (substituting into the reveal invariant using `secret = c_0+c_1+c_2` confirms it holds at every ring position), and empirically against the real compiled binary (a single-client, single-element share converted via this formula and revealed through all 3 real executables, on the first attempt).

7.3 The `operator+` pitfall

The obvious design — write N clients' shares into the input file and let the compiled program sum them itself with `Additive_Share::operator+`, mirroring how `programs/functions/log_reg.hpp` accumulates a gradient across data points — **does not work** for summing independently-shared secrets. `Additive_Share::operator+` for `PROTOCOL==2` calls `Replicated_Share::Add(b, OP_SUB)`, which keeps the **left operand's `x`** unchanged and **subtracts** the right operand's `a`. That is correct for combining a running share with a *locally-derived delta* (which is how `log_reg.hpp` actually uses it), but it is emphatically not addition of two otherwise-unrelated secrets. This was confirmed with a minimal diagnostic during development: sharing 10 and 20 independently and adding them via hpmpc's own `+` revealed `-10` consistently across all 3 parties, not `30`.

The fix: summing replicated shares of independent secrets is genuinely free, local arithmetic — the entire point of additive-style sharing — as long as **both** the `x` and `a` fields are added elementwise. Since `Replicated_Share`'s fields are private with no public getters, the hpmpc program never attempts this in C++ at all. Instead, **all cross-client summing happens in Python**, in `backend_hpmpc.py`, before the input

file is ever written: every selected client's `(x, a)` pair is added (`mod 2**bitlength`) into one running total per model-weight element. The compiled program therefore never sees more than one already-combined `(x, a)` pair per element — it only ever has to reveal it. This turned out to be *simpler* than a client-count-aware C++ summing loop, not just safer.

7.4 The compile-time double-instantiation complication

`protocol_executer.hpp` instantiates every `FUNCTION` **twice**: once during an init phase (`Share = Replicated_init<DATATYPE>`, a lightweight stub used only to size communication buffers for the live phase — its output is explicitly discarded) and once for the real live phase (`Share = Replicated_Share<DATATYPE>`). `Replicated_init` has no `(Datatype, Datatype)` constructor, so directly constructing a share from a raw `(x, a)` pair only compiles for the live instantiation. `fedavg_secure_aggregation.hpp` guards that construction (and the final output-file write) with `if constexpr (std::is_same_v<Share, Replicated_Share<DATATYPE>>)` — discarded entirely, not merely skipped, for the init-phase instantiation, which is what makes it compile at all. Both phases still run the *same* sequence of `prepare_reveal_to_all`/`communicate`/`complete_reveal_to_all` calls (one per element), which is what keeps the init phase's buffer-size bookkeeping consistent with what the live phase actually sends over the wire.

7.5 File contract

```
input file:  uint32 elements_per_client
             then that many (uint64 x, uint64 a) pairs -- the PRE-SUMMED
             share of the weighted total, one pair per flattened
             model-weight element (all layers concatenated, sorted by
             layer_name for a deterministic order).

output file: uint32 elements_per_client
             then that many uint64 values -- the revealed raw ring
             representation of the sum (decode via FixedPointCodec).
```

Both paths are passed via `SECURE_AGG_INPUT_FILE`/`SECURE_AGG_OUTPUT_FILE` environment variables, not CLI args — hpmpc's own CLI args are reserved for peer IP addresses: `./run-P{party}.o <ip1> <ip2>` (positional, peers only, self excluded, **sorted by real party index ascending** — do not reorder; `HpmpcBackend.start()` sorts `peer_endpoints` for exactly this reason).

A real, independently-hit deployment bug: hpmpc's own C++ socket layer parses these peer-IP CLI arguments as literal dotted-quad addresses — it has **no DNS resolution of its own**, unlike gRPC/Python elsewhere in this codebase. A Docker Compose service name like `secure_agg_party1` crashed it outright (`terminate called after throwing 'std::runtime_error': Invalid address: secure_agg_party1`) during the Phase 4 real end-to-end run. Fixed by having `backend_hpmpc.py` call `socket.gethostbyname(peer.host)` itself, fresh on every round (not cached at `start()`-time), before constructing the CLI args — this way a peer container that restarts mid-deployment with a new IP doesn't leave the backend pointed at a stale address.

7.6 The config-consistency check

`bitlength`/`frac_bits` live in two independent places with no automatic way to keep them in sync: a party's YAML config (read by Python's `FixedPointCodec`) and hpmpc's compile-time `BITLENGTH`/`FRACTIONAL` macros. `build_fedavg_secure_aggregation.sh` writes `executables/fedavg_secure_aggregation.build_metadata.json` recording what the binaries were actually compiled with. `HpmpcBackend._check_config_consistency()`, called from `start()` (so a mismatch fails a party at **startup**, never mid-round), compares this metadata against the party's configured codec and raises `RuntimeError` on any mismatch, or if the metadata file or an expected executable is simply missing.

7.7 The Docker build, and the errors hit along the way

`docker/ Dockerfile.secure_agg_party.hpmpc` is a multi-stage build: `hpmpc-build` compiles the party executables from the hpmpc source (mounted via Compose's `additional_contexts`, since hpmpc lives in a sibling directory outside this repo's build context); `runtime` installs the Python dependencies and copies in just the compiled binaries. **Both stages deliberately use the same base distro** (`ubuntu:22.04`) — copying a compiled C++ binary into a *different* base image risks a glibc version mismatch (the runtime image's glibc must be at least what the binary was linked against), and keeping both stages identical sidesteps that entirely.

Getting this working surfaced a sequence of real, unglamorous build failures, each worth knowing about if this Dockerfile ever needs to change again:

1. **Missing `git`** in the minimal runtime stage broke a pip source build that needed it. Fixed: added `git build-essential` to the apt install.
2. **Ubuntu 24.04's default Python (3.12) has no prebuilt wheel** for the pinned `numpy==1.24.3`, and falling back to a source build hit `AttributeError: module 'pkgutil' has no attribute 'ImpImporter'` (removed in Python 3.12, an old setuptools bug). Fixed by switching **both** stages to `ubuntu:22.04` (default Python 3.10, which does have a wheel; this also incidentally avoids the glibc-mismatch risk mentioned above, since 22.04 ships gcc-11 instead of 24.04's gcc-12).
3. **`--break-system-packages`** (a pip flag for Debian/Ubuntu's PEP 668 guard, needed on 23.04+/Debian 12+) is not recognized on 22.04's older pip and was rejected as an unknown option. Fixed by removing it.
4. **`psutil` failed to compile its C extension** (`Python.h: No such file or directory`). Fixed by adding `python3-dev` to the apt install.

7.8 How this was verified without a working native build environment

The macOS development host's Xcode Command Line Tools were missing libc++ headers (confirmed with a minimal `#include <algorithm>` failing identically outside hpmpc entirely), so all verification happened inside an Ubuntu Docker container, in increasing order of realism:

1. A single-client, single-element share, converted via the `(x, a)` formula (Section 7.2) and revealed through the 3 real compiled binaries — isolated the share-format derivation from any summing logic, and passed on the first attempt.
2. A 3-client test summed via hpmpc's own `Additive_Share::operator+` inside the C++ program — this is what surfaced the `operator+` pitfall (Section 7.3): revealed values were inconsistent garbage across parties.
3. The same 3-client test, pre-summed in Python instead (the fix) — all 3 parties revealed the exact expected elementwise sums.
4. The same test using the *actual* `replicated3pc.py` and `fixed_point_codec.py` modules (not reimplemented test logic), with negative and fractional float values — confirmed correct decode.
5. The actual `HpmpcBackend` class (not just the raw file format), with multiple clients and multiple differently-shaped layers, run through real subprocesses via `asyncio` — matched the expected weighted raw sum exactly.
6. Finally, the full real end-to-end MNIST + LeNet5 training run (Section 8.2) — genuine production-shaped usage, not a synthetic test.

What was actually validated

8.1 Automated test suite

73 tests, spanning three tiers:

- `unit` (fast, no Docker, no hpmpc toolchain): sharing-math round trips and property tests (`hypothesis`), fixed-point codec boundary tests, the aggregator's control-flow logic against a mocked orchestrator, the hpmpc backend's file-format glue and config-consistency check against a **mocked subprocess** (no real binary needed), the party server's share-buffering and round-triggering logic, and regression tests for every bug in Section 2.4.
- `integration` (in-process asyncio, no Docker): a full round through 3 real in-process party server instances plus a fake client, matched against plaintext FedAvg; a fault-injection test that kills a party mid-round and confirms the round fails promptly with state cleaned up; an aggregator-level equivalence test comparing `fedavg` and `secure_mpc` directly.
- `e2e` (docker-compose, opt-in, not required every commit): the full 3-process party cluster brought up over real Docker networking, health- checked, and driven through a real round.

Re-run after every fix in this project (including the four real bugs in Section 2.4): **73 passed, 0 failed.**

8.2 The real end-to-end MNIST + LeNet5 training run

At the user's explicit choice, validation went beyond the aggregator-level equivalence test to a full real training run: 3 rounds, LeNet5, real MNIST data partitioned across 3 clients plus a validation split (`docker/prepare_mnist_data.py`), driven through the entire real stack — real Docker containers, real gRPC, and for the `secure_mpc` run, the **real compiled hpmpc binaries**, not a simulator.

Run	Round 1	Round 2	Round 3
`fedavg` (plaintext baseline)	~92%	~94%	~96%
`secure_mpc` (real hpmpc, 3 containers)	71.95%	97.35%	97.7%

Both runs converge to comparable final accuracy, confirming the secure aggregation pipeline computes the correct result end to end with real cryptographic computation, not just in unit tests against synthetic tensors.

8.3 Fault injection and recovery, verified live

Beyond the automated fault-injection integration test (Section 8.1), the same scenario was exercised against the real live Docker cluster:

1. `secure_agg_party1` was stopped (`docker compose stop`) mid-deployment, simulating a real party crash.
2. A fresh session was submitted against the now-degraded (2-of-3) cluster. Every client's `SubmitShare` call to the stopped party failed almost instantly with a DNS resolution error (Docker tears down a stopped container's DNS entry), which propagated as a `grpc.RpcError` and was absorbed by Flotilla's **pre-existing** "client dropped" handling path — confirming the documented claim that a party outage produces a clean, fast per-round failure, not a silent hang.
3. `secure_agg_party1` was restarted and confirmed healthy via its own startup logs.
4. State was reset (Redis flushed, `flo_server`/clients restarted for a clean slate) and a **fresh, full** `secure_mpc` session was submitted against the now fully-healthy 3-party cluster.
5. That session completed successfully (`Session ... finished`), confirming the cluster **fully recovers** once the previously-stopped party is healthy again — this was not just asserted in a doc, it was demonstrated live.

How to add a new MPC backend: a worked, step-by-step guide

This is the concrete answer to "what would it actually take to plug in a different MPC library" — the question the entire genericity requirement in Section 1.4 exists to make cheap to answer well. Nothing below requires

touching `flo_server.py`, `server_session_manager.py`, `party_server.py`, `aggregator_secure_mpc.py`, or any client-side training code — that is the whole point of the abstraction boundary in Section 4.

Suppose you want to add a backend for a hypothetical library, `acmempc`, that (unlike hpmpc) ships Python bindings and runs as a long-lived daemon per party rather than spawning a fresh process every round — a genuinely different integration shape from hpmpc's, deliberately chosen for this example to stress-test the abstraction rather than just re-deriving hpmpc's approach.

Step 1 — Decide whether you need a new `SecretSharingScheme` too

If `acmempc` uses the same (2,3)-replicated additive sharing hpmpc does, you can reuse `sharing_schemes/replicated3pc.py` unchanged — sharing schemes and backends are independent axes (Section 4.1/4.3). If it uses a genuinely different scheme (e.g. Shamir's), implement a new `SecretSharingScheme` subclass in `sharing_schemes/acmempc_shares.py`, exposing it as `SCHEME_CLASS` (follow `shamir_stub.py` as a structural template — it exists precisely to prove the ABC doesn't assume replicated-sharing specifics). No other file needs to change; `load_sharing_scheme.py` picks it up automatically once `sharing_scheme: acmempc_shares` appears in config.

Step 2 — Write the backend module

Create `src/server/secure_agg/ backends/backend_acmempc.py`:

```
from server.secure_agg.backends.base import SecureAggregationBackend

class AcmeMpcBackend(SecureAggregationBackend):
    backend_id = "acmempc"

    def __init__(self, party_index, num_parties, codec, daemon_socket_path, **kwargs):
        self.party_index = party_index
        self.num_parties = num_parties
        self._codec = codec
        self._daemon_socket_path = daemon_socket_path
        self._client = None   # acmempc's own Python binding client, opened in start()

    async def start(self, peer_endpoints):
        # Unlike hpmpc (which just remembers peer IPs for later, since it
        # opens fresh sockets per round), a daemon-backed library's start()
        # is where you'd actually open the persistent connection:
        import acmempc
        self._client = acmempc.connect(self._daemon_socket_path, peers=[
            (p.party_index, p.host, p.port) for p in peer_endpoints
        ])

    async def run_aggregation_round(self, round_id, shares, tensor_specs, timeout_s):
        # Sum shares locally first -- this step is IDENTICAL across every
        # backend, since it's just the sharing scheme's local `add`, with
        # zero dependency on acmempc. Only the reveal step below is
        # library-specific.
        summed = _sum_shares_locally(shares, tensor_specs)   # same helper backend_hpmpc.py uses
        raw_result = await self._client.reveal(round_id, summed, timeout=timeout_s)
        return _decode_to_tensor_dict(raw_result, tensor_specs, self._codec)

    async def stop(self):
        if self._client is not None:
            await self._client.close()
```

```
BACKEND_CLASS = AcmeMpcBackend
```

Notice `run_aggregation_round` never spawns a subprocess, never touches a file, and never has an `if constexpr`-style compile-time branch anywhere — none of hpmpc's specific integration shape is visible here at all, because none of it was ever part of the `SecureAggregationBackend` contract to begin with.

Step 3 — Register the constructor dispatch

`flo_secure_agg_party.py`'s `_construct_backend()` is the **one** place that knows each backend needs different constructor arguments (the simulator needs a `bind_host`/`bind_port` for its peer gRPC service; hpmpc needs an `executable_dir`/`tmp_dir`; `acmempc` here needs a `daemon_socket_path`). Add one branch:

```
if backend_type == "acmempc":
    return backend_module.BACKEND_CLASS(
        party_index=party_index,
        num_parties=num_parties,
        codec=codec,
        daemon_socket_path=per_backend_config["daemon_socket_path"],
    )
```

Nothing else in `flo_secure_agg_party.py`, `party_server.py`, or anywhere above the backend boundary changes.

Step 4 — Add the config surface

Extend `secure_agg_party_config.yaml`'s `backend:` block:

```
backend:
  type: acmempc
  acmempc:
    daemon_socket_path: /run/acmempc/party0.sock
```

Step 5 — Write unit tests against a mocked client, exactly like `backend_hpmpc.py`'s tests do against a mocked subprocess

`tests/unit/test_backend_acmempc.py`: mock `acmempc.connect/reveal`, verify the local-sum-before-reveal logic, verify `start()`/`stop()` lifecycle, verify config validation. No real `acmempc` daemon or binary needed for this tier — mirror `tests/unit/test_backend_hpmpc.py`'s structure directly.

Step 6 — Prove it against the plaintext reference, then wire it into docker-compose

Run the existing `tests/integration/ test_full_secure_agg_round.py`-style scenario against your new backend (swap the fixture's backend construction) to confirm it matches `fedavg`'s math within tolerance — this is the same check Phase 2's `SimulatorBackend` and Phase 3's `HpmpcBackend` both had to pass before being trusted. Then add a `docker/ Dockerfile.secure_agg_party.acmempc` and a docker-compose service variant, following `Dockerfile.secure_agg_party.hpmpc`'s structure (Section 7.7) if `acmempc` needs its own build step, or `Dockerfile.secure_agg_party`'s (the plain, single-stage image) if it's pure Python/pip-installable.

What you did *not* have to touch

`flo_server.py`, `server_session_manager.py`, `party_server.py`, `aggregator_secure_mpc.py`, `client_grpc_manager.py`, `client_secure_agg_manager.py`, `secure_agg.proto`, or any client-side training code. Every one of those operates purely in terms of the ABCs in Section 4 — that boundary is precisely what made this exercise a new-file-plus-one-dispatch-branch change instead of a redesign, which was the whole point of the genericity requirement stated at the very start of this project.

Current limitations

Stated plainly, so nobody assumes more than this project actually delivers. None of these are silently swept under the rug — each is tracked in `docs/secure_aggregation/threat_model.md` and/or `hpmpc_backend.md`, and several are backed by an actual test proving the claim rather than just an assertion.

1. **No fault tolerance.** Losing 1 of the 3 party processes mid-round blocks that round — the *live* MPC protocol genuinely needs all 3 parties participating, unlike offline reconstruction (which only needs 2 of 3 shares). Confirmed both by an automated fault-injection test and by a live exercise (Section 8.3): the failure is clean and fast (not a hang), and the cluster recovers fully once the party returns — but training simply cannot progress while a party is down.
2. **Semi-honest / honest-majority only.** No protection against a party that actively deviates from the protocol on purpose. If 2 of the 3 parties collude, replicated (2,3) sharing is broken by construction (2 shares reconstruct the secret) — the 3 parties must be run by genuinely independent, non-colluding operators for the design's guarantee to mean anything. A malicious-secure upgrade (e.g. hpmpc's 4-party Tetrad protocol) is a documented future path, not built here.
3. **No transport encryption by default.** Every gRPC channel in this project (`SecureAggPartyService`, `SecureAggPeerService`) is, like the rest of Flotilla today, plaintext (`grpc.insecure_channel`) — a pre-existing gap in Flotilla, not a regression introduced here, but it does mean shares travel unencrypted over the control plane unless hardened separately. hpmpc's own `USE_SSL=0` (Section 7.1) is likewise a dev/test choice, not production-hardened.
4. **What is protected is narrower than "everything."** A client's per-round local model update AND (in `secure_mpc` sessions) its dataset size are hidden — the party cluster reveals only the round's *total* dataset size, never an individual client's. Training metrics/loss, round participation, the resulting global model, and model architecture/hyperparameters are all still plaintext — see `threat_model.md` for the full breakdown. This inherits the same degenerate case the model update itself has: with only one client checked in for a round, the revealed "total" IS that client's exact dataset size — a property of sum-based aggregation privacy generally, not a bug here.
5. **Clients are trusted to secret-share honestly.** A malicious client could submit garbage shares — this is the same data-quality/poisoning exposure Flotilla's plaintext path already has today (a client can already submit a garbage plaintext update), not a new attack surface.
6. **`round_id`-based replay/mix-up protection is best-effort, not a security guarantee.** It catches accidental cross-round mix-ups (bugs, misconfiguration, stale retries), not an adversarial party deliberately replaying old shares.
7. **`verify_party_agreement` is a correctness sanity check, not a security mechanism.** A colluding or buggy majority of parties can still agree on a wrong answer and pass this check.
8. **Process-per-round overhead, unmeasured.** hpmpc's own execution model spawns a fresh executable every round; for very frequent, small rounds this has more overhead than a persistent daemon would. Not measured or optimized in this project — the exact number depends heavily on model size and deployment network latency, so no fixed figure is claimed.
9. **Only `PROTOCOL=2` (Replicated 3PC) is wired up.** Swapping to hpmpc's own higher-performance `PROTOCOL=5` ("Trio") variant, or a genuinely different 4-party protocol, is a documented future path (Phase 5) — the share-format work in Section 7.2 is specific to Replicated's `(x, a)` layout and would need to be redone for a different protocol's representation.
10. **A latent numpy-version wire-format coupling.** `PartyShare.payload` for `replicated3pc` is pickled numpy arrays, and numpy's pickle format embeds version-specific internals (`numpy._core` on numpy>=2.0 vs `numpy.core` on numpy 1.x) — a client and a party process on incompatible numpy major versions would fail to interoperate even though nothing in this code changed. Today's pinned `numpy==1.24.3` across every component's `requirements.txt` avoids this in the checked- in deployment, but it's a latent fragility if any component's pin ever drifts independently. A documented hardening candidate: serialize `PartyShare` payloads as raw bytes plus explicit shape/dtype (already carried on the wire anyway) instead of `pickle.dumps(ndarray)`.

11. **No automated health-monitoring loop.** `HealthCheck` exists on every party (Section 5.4/6.5) but nothing currently polls it automatically — it's a manual/scripted debugging aid today, not wired into any alerting.
12. **The `SimulatorBackend`'s share cache grows unboundedly.** It never evicts a finished round's cached local sum (a real race was hit and fixed during development by *not* popping early — Section 1.3, Phase
    2) — acceptable for a reference backend meant to validate the topology, not to run indefinitely; a production-shaped backend would need a TTL-based sweep independent of any single round's completion.

Appendix

A.1 Command cheat sheet

```
# Run the fast test tiers
pytest -m "unit or integration"

# Build the real hpmpc executables (inside a Linux/Docker environment)
cd hpmpc && scripts/build_fedavg_secure_aggregation.sh 64 13

# Bring up the party cluster only (hpmpc-backed by default)
cd flotilla/docker
docker compose up -d --build secure_agg_party0 secure_agg_party1 secure_agg_party2

# Bring up the full example stack (redis, mqtt, 3 parties, server, 3 clients)
python3 prepare_mnist_data.py
docker compose up -d --build

# Submit a secure_mpc training session
python3 ../flo_session.py ../config/training_config.secure_mpc.example.yaml \
    --federated_server_endpoint localhost:12345

# Tear everything down
docker compose down -v
```

A.2 Directory map (new/changed paths only)

```
flotilla/
|-- src/
|   |-- flo_secure_agg_party.py                        [new]
|   |-- client/
|   |   |-- client_secure_agg_manager.py                [new]
|   |   |-- client_grpc_manager.py                      [modified]
|   |   `-- client_manager.py                           [modified]
|   |-- server/
|   |   |-- server_session_manager.py                   [modified]
|   |   |-- aggregation/aggregator_secure_mpc.py         [new]
|   |   |-- aggregation/aggregator_fedavg.py             [bugfix]
|   |   `-- secure_agg/                                  [new package]
|   |       |-- sharing_schemes/{base,replicated3pc,shamir_stub}.py
|   |       |-- backends/{base,backend_simulator,backend_hpmpc}.py
|   |       |-- load_sharing_scheme.py
```

```
|   |       |-- load_backend.py
|   |       |-- fixed_point_codec.py
|   |       |-- party_server.py
|   |       `-- party_orchestrator_client.py
|   `-- proto/secure_agg.proto (+ generated stubs)      [new]
|-- config/
|   |-- secure_agg_party_config.yaml                    [new]
|   `-- training_config.{fedavg,secure_mpc}.example.yaml [new]
|-- docker/
|   |-- Dockerfile.secure_agg_party[.hpmpc]             [new]
|   |-- secure_agg_party_entrypoint.sh                  [new]
|   |-- prepare_mnist_data.py                           [new]
|   `-- docker-compose.yaml                             [modified]
|-- docs/secure_aggregation/                             [new]
`-- tests/{unit,integration,e2e}/                        [new]

hpmpc/
|-- programs/functions/fedavg_secure_aggregation.hpp     [new]
|-- protocol_executer.hpp                                [modified: +1 dispatch case]
`-- scripts/build_fedavg_secure_aggregation.sh           [new]
```

A.3 Glossary

- **MPC (Multi-Party Computation)**: a family of cryptographic protocols letting several parties jointly compute a function over their private inputs while revealing nothing except the output.
- **Secret sharing**: splitting a value into pieces ("shares") distributed across parties such that no subset below a threshold learns anything about the value, but a sufficient subset can reconstruct it.
- **Replicated (2,3) secret sharing**: the specific scheme used here — a value is split into 3 additive components, and each of the 3 parties holds 2 of them (a different pair per party), so any 2 parties together can reconstruct.
- **Semi-honest (honest-majority)**: a threat model where parties are assumed to follow the protocol correctly but might passively try to learn secrets from what they legitimately see.
- **Fixed-point encoding**: representing a real number as a scaled integer (`round(x * 2**frac_bits)`) so that secret-sharing/MPC arithmetic, which operates over integer rings, can approximate real-number computation.
- **Reveal**: the step in an MPC protocol where parties exchange enough information to reconstruct a value in the clear — the only point at which plaintext information is produced.
- **`PROTOCOL`, `FUNCTION_IDENTIFIER`, `BITLENGTH`, `FRACTIONAL`, `DATTYPE`, `RANDOM_ALGORITHM`, `USE_SSL`**: hpmpc's own compile-time (`-D`) macros selecting, respectively, which MPC protocol, which compiled function, the integer ring size, the fixed-point fractional-bit count, the underlying machine word type, the PRNG implementation, and whether transport is TLS-encrypted.
- **`round_id`**: `f"{session_id}:{round_idx}"`, carried on every secure-aggregation RPC as a replay/mix-up guard (not a security boundary — see Section 10, item 6).